

## 阿蛮

特判  $n = 1$ , 下文默认  $n > 1$ 。

设  $f_{x,y}$  表示  $\sum a_i = x, a_1 = x - y$  的概率。不难写出递推：

```
f[n][n-1]=1;
f(i,1,m)f(j,1,m)if(l=f[i][j]%mo){
    l=l*ksm(i)%mo;//i的逆元
    f[i+1][j+1]+=l*j%mo;
    f[i+1][j]+=l*(i-j)%mo;
}
```

到此有 50 ~ 60 分。

观察到在定义范围内  $f_{x,y} = \frac{\binom{y-1}{n-2}}{\binom{x-1}{n-1}}$ 。

这是有组合意义的。我们为每个人（观众和舞者）编号（舞者从左到右为  $1 \sim n$ ）。一个人加入时，将他插入到他选择的观众后面。

$\sum a_i = x$  的时候所有情况是一个  $1 \sim x$  的排列，满足 1 是开头，且  $1 \sim n$  相对有序。所有符合要求的情况出现概率相同。

1 后面有  $x - 1$  个人，其中有  $n - 1$  个是舞者，故总方案数为  $\binom{x-1}{n-1} \times (x - n)!$ 。

2 后面有  $y - 1$  个人，其中有  $n - 2$  个是舞者，合法方案数为  $\binom{y-1}{n-2} \times (x - n)!$ 。

$(x - n)!$  约去后即  $f_{x,y} = \frac{\binom{y-1}{n-2}}{\binom{x-1}{n-1}}$ 。

则最终答案为：

$$\sum_{x=n+1}^{n+m} \sum_{y=1}^x f_{x,y} \times \max(x - 2y, 0) = \sum_{x=n+1}^{n+m} \frac{1}{\binom{x-1}{n-1}} \sum_{y=1}^{\lfloor \frac{x}{2} \rfloor} \binom{y-1}{n-2} \times (x - 2y)$$

前缀和维护一下即可。复杂度  $O(n + m)$ 。

## 菟园

贴一个形式化题意。

强制在线， $n$  次操作，维护一个字符串  $S$ ，支持：

1. 往末尾加入一个字符。
2. 查询有多少  $S$  有多少子串与  $S_{[1,k]}$  相同。

根据 KMP 的性质，我们建出 fail 树 ( $i \rightarrow next_i$  连边得到的内向树)； $S_{[1,k]} = S_{[i-k+1,i]}$ ，当且仅当在 fail 树上  $k$  是  $i$  的祖先，所以查询就是询问点  $k$  的子树大小。

我们需要维护一棵树，支持在线地插入叶子、查询某个点的子树大小。

不加思考地，这可以用 LCT 在  $O(n \log n)$  时间内实现。由于过于复杂这里不给出参考代码。

可以用一个经典分块技巧 ([P3203 \[HNOI2010\] 弹飞绵羊](#)) 做到  $O(n\sqrt{n})$ 。

如果觉得题解过于复杂，可以去 P3203 学习一下该技巧，或配合 std 阅读。

首先这个问题还可以简单地转化一下，我们需要维护一棵树，在线完成下述操作：

1. 加入一个叶子（其点权为 0），让从这个叶子到根所有点的点权 +1。
2. 查询某个点的点权（每个点的点权就是其子树大小）。

钦定根的父亲  $fa_1 = 0$ 。考虑最暴力的做法，我们加入一个叶子  $i$  后不断跳到其父亲  $fa_i$ ，并更新点权。

数据随机的情况下树高普遍不高，故此做法可以通过 A 性质。

注意到，点  $i$  的父亲编号  $fa_i$  一定满足  $fa_i < i$ （或者说  $next_i < i$  也可以）。

考虑优化，将整个序列分块（块长为  $B$ ）。

处理出每个点  $i$  一直跳父亲，首次跳出所在块会到达点  $g_i$ （每次加入  $i$  后计算  $g_i$  即可，复杂度为  $O(1)$ ）。

同时维护两个序列  $a_i, b_i$ 。 $a_i$  表示点权， $b_i$  表示懒标记（从  $i$  一直跳  $fa_i$  直到跳出所在块，经过所有点的  $a_i$  需要加上  $b_i$ ）。

加入叶子  $i$  时，将  $b_i + = 1$ ，然后令  $i \leftarrow g_i$ ；重复操作直到  $i = 0$ 。由于每次一定会跳出当前块，故此部分复杂度为  $O(\frac{n}{B})$ 。

查询节点  $k$  时，设其所在块为  $[l, r]$ ，我们从后往前遍历  $i = \{r \rightarrow l\}$ ，依次执行以下操作：

1. 令  $a_i + = b_i$ 。
2. 若  $l \leq fa_i$ ，则  $b_{fa_i} + = b_i$ 。
3. 令  $b_i = 0$ 。

最后输出  $a_k$  即可，这部分复杂度是  $O(B)$ 。

平衡复杂度  $B = O(\sqrt{n})$  时，总复杂度为  $O(n\sqrt{n})$ ，可以通过。

## 玉真

---

考虑  $k = 2$  怎么做。

由于  $c \oplus 1$  (异或,  $c = 0/1$ ) 与下面的公式套用有点阴间, 记为  $1 - c$ 。

发现  $w(1, i)$  没啥规律, 考虑  $w(0, i)$  长什么样:  $w(0, 0) = 0$ ,  $w(0, 1) = 01$ ,

$w(0, 3) = 0110$ ,  $w(0, 7) = 01101001$ ,  $w(0, 15) = 0110100110010110$ 。

不难发现, 因为长度每翻一倍就在原先每个数的二进制中都多了一个 1, 就是取反拼在后面。

记  $w_{t,0} = w(0, 2^t - 1)$ ,  $w_{t,1} = w(2^t, 2^{t+1} - 1)$ , 有  $w_{t,c} = w_{t-1,c} + w_{t-1,1-c}$ 。

回到问题, 套路地对  $p_i$  建 AC 自动机, 将  $S$  跑一遍, 每个串的答案就是对应点的 fail 子树到达次数和。

肯定不能跑  $S$ , 找点性质; 发现  $w_{t,c}$  的结构与线段树很像, 可以把  $w(l_i, r_i)$  拆成  $O(\log V)$  个  $w_{t,c}$ 。

可以记  $f[x][t][c]$  表示从 AC 自动机上节点  $x$  开始将  $w_{t,c}$  跑一遍后所在的节点。

根据  $w_{t,c}$  的转移有  $f[x][t][c] = f[f[x][t-1][c]][t-1][1-c]$ 。

$x$  开始将  $w_{t,c}$  跑一遍, 经过点的到达次数要加  $g[x][t][c]$ 。

全部跑完后将  $g[x][t][c]$  加到  $g[x][t-1][c]$  和  $g[f[x][t-1][c]][t-1][1-c]$ 。

我们只用将到达次数  $w[f[x][0][c]] + g[x][0][c]$  即可。

fail 子树到达次数和直接 dfs 即可。

---

$k > 2$  怎么办呢?

记  $w_{t,i} = w(i \times k^t, (i+1) \times k^t - 1)$ , 同样有:

$$w_{t,c} = w_{t-1,c} + w_{t-1,(c+1) \bmod k} + \dots + w_{t-1,(c+k-2) \bmod k} + w_{t-1,(c+k-1) \bmod k}$$

同上对  $p_i$  建 AC 自动机, 将  $S$  跑一遍。还是类似线段树的递归把  $w(l_i, r_i)$  拆成  $O(\log V)$  个  $w_{t,c}$ 。AC 自动机上倍增算答案。

总复杂度  $O((n + \sum |p_i|) \log V)$ 。

# 乘烟

---

题里都说了，我们姑且将每个点的度数不超过  $n$  视为有解的充要条件。

很难不发现，如果有度数为  $n$  的点，那么有以下做法：

1. 令度数最大的点为根（**多个必须优先选上次选的**），若  $n = 2$ ，将点  $1, 2$  染成颜色  $1, 2$ ，结束。
2. 找到一个和根相连的叶子，一个不和根相连的叶子，将它们染成颜色  $n + 1$  并删去。回到 1。

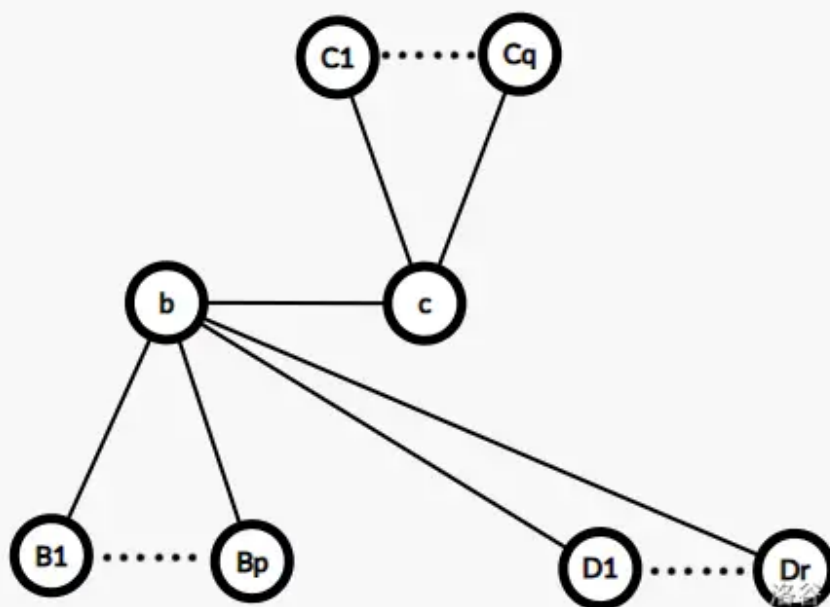
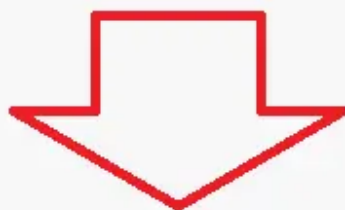
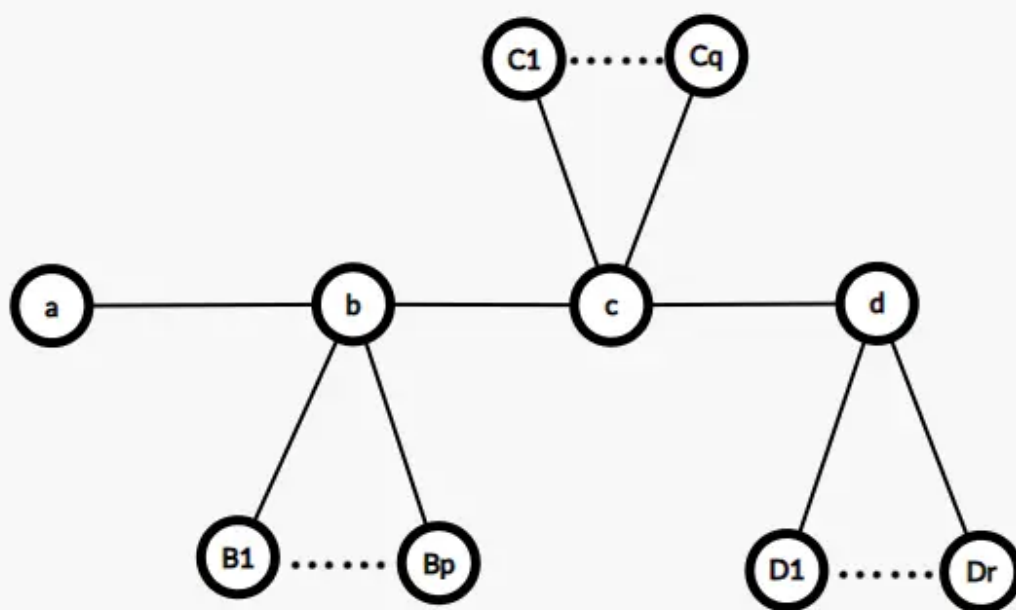
否则，我们考虑一些奇怪的做法。不断把问题规模缩小直到存在度数为  $n$  的点。

下文默认所有点度数小于  $n$ 。

称元组  $(a, b, c, d)$  是好的，当且仅当  $b$  点的度数  $+ d$  点的度数  $< n + 2$ 。

**注意到，若  $b, d, f, h$  互不相同，则元组  $(a, b, c, d)$  和元组  $(e, f, g, h)$  中至少有一个是好的。**

对于一个好元组  $(a, b, c, d)$ ，若  $a$  是叶子，边  $(a, b), (b, c), (c, d)$  都存在，可以将点  $a, d$  染成颜色  $n + 1$  并删去，再将原来和  $d$  相连的点（ $c$  除外）连向  $b$ 。新的树限制不弱于之前的限制，且依然满足有解。



当没有度数为  $n$  的点时，选择两个满足  $\text{dis}(x, y) > 2$  的叶子  $x, y$ 。

设数组  $p_1, p_2, \dots, p_m$  表示  $x \sim y$  路径经过的点 ( $m = \text{dis}(x, y) + 1, p_1 = x, p_m = y$ )。

若  $m \leq 5$  时，可以直接将  $x, y$  染成颜色  $n + 1$  并删去。

当  $m > 5$  时, 以下必然有一个条件成立:

1. 元组  $(p_1, p_2, p_3, p_4)$  是好的。
2. 元组  $(p_m, p_{m-1}, p_{m-2}, p_{m-3})$  是好的。
3.  $m = 7$ , 且存在一个叶子  $z$  与  $p_4$  相连。

情况 1, 2 构造已经写过了; 情况 3 注意到此时  $dis(x, z) = 4$ , 将  $x, z$  染成颜色  $n + 1$  并删除即可。

**注意, 情况 3 要求不存在节点度数为  $n$ , 故这里提到的做法必须和特殊性质拼起来得到正解。**

由于要计算  $dis$ , 找到  $p_{2 \sim 4}, p_{m-3 \sim m-1}$ , 加边和删边, 考虑使用 LCT 解决。配上启发式合并的思想可以做到  $O(n \log^2 n)$ 。代码太丑不放了。理论可以做到  $O(n \log n)$  但是细节很多。

---

当然如果你认为这题正解是 LCT 那你就和我一样输完了。

依然是两个满足  $dis(x, y) > 2$  的叶子  $x, y$ 。不难快速找到任意一条从  $x$  出发的, 包含 4 个点的链  $(x, x_1, x_2, x_3)$ ; 同理可以找到链  $(y, y_1, y_2, y_3)$ 。

如果这两条链有交, 那么我们就找到了数组  $p_1, p_2, \dots, p_m$ , 套用上面的做法。

否则元组  $(x, x_1, x_2, x_3)$  和元组  $(y, y_1, y_2, y_3)$  至少有一个是好的, 完结撒花。

如果你闲的无聊可以用线段树合并维护边集做到  $O(n \log n)$ , 但是 std 写的是启发式合并 set,  $O(n \log^2 n)$  跑 0.9s。

---

上述做法有许多地方可以用不同的特判解决。题解思路仅供参考。

如果你有更简洁的做法欢迎讨论。

